Machine Translated by Google





Internship in System Programming

Attempt 2

Scheduler / Stack

Chair of Computer Science 11 - RWTH Aachen

November 19, 2025

Commit: Unknown

Machine Translated by Google

Table of contents

2 Scheduler / Stack **3**
2.1 Basics 4
2.1.1 Programs and processes 4
2.1.2 Scheduler 4
2.1.3 Critical areas 7
2.1.4 Stack 7
2.1.5 Memory allocation 10
2.1.6 Storing Strings 10
2.2 Homework 12
2.2.1 System Programming Operating System – SPOS 12
2.2.2 Error handling. 14
2.2.3 Scheduler in SPOS 15
2.2.4 Management of programs and processes 16
2.2.5 From program to process 16
2.2.6 Implementation of the scheduler. 18
2.2.7 Implementation of critical areas. 24
2.2.8 Detection of stack inconsistencies. 25
2.2.9 Opening the Task Manager 26
2.2.10 Summary 28
2.3 In person 30
2.3.1 Test tasks 30
2.3.2 Application tasks 30
2.4 Pin assignment 31

This document is part of the accompanying materials for the *System Programming internship.* All documents required for this internship are available for download in the Moodle learning environment at https://moodle.rwth-aachen.de . The following email address is available for criticism, suggestions, or feedback:

support.psp@embedded.rwth-aachen.de

Machine Translated by Google

2 Scheduler / Stack

The learning objective of this experiment is to understand the following relationships:

- How a scheduler works
- Management of programs and processes
- Management of operating system and process stacks
- Management of critical areas
- Fault detection

Notice

The documents for the System Programming internship consist of two parts:

- **Basics**
- **Homework** and **In-person instruction**

To complete the tasks, you will need some of the theory presented in the introductory chapter. It is also recommended to consult the *accompanying document* if anything is unclear.

Machine Translated by Google

2.1 Basics

The following section introduces the concepts of "programs and processes", "scheduler", "critical areas" and "stack".

2.1.1 Programs and Processes

An operating system distinguishes between *programs* and *processes.* A program is a finite sequence of operations. A process is an instance of a program, managed by the operating system within its *runtime context .*
The runtime context is the sum of the processor registers that represent the execution state of a process at any given time. To allow a program to be started multiple times in an operating system, a separate process with its own runtime context is created for each program instance. Thus, each program instance is executed independently. Such an operating system is also called *a multitasking operating system.*

Generally, a distinction is made between two types of programs: terminating and non-terminating programs. For the sake of simplicity, this experiment will only deal with non-terminating programs, and thus also processes. The treatment of terminating programs/processes will be explained in Experiment 3.

2.1.2 Scheduler

A scheduler is used in multitasking operating systems to manage processor time and to transparently switch between processes. In this context, transparency means that processor time and stack memory are ideally available to each process in such a way that the process cannot distinguish whether it has exclusive access to the system or not.

A scheduler can, for example, interrupt the currently running program at periodic intervals to execute another process. When a process is interrupted by the scheduler, the runtime context of that process must be saved. Once the process is reactivated and allocated processing time at a later time, the corresponding runtime context must be restored. This ensures that the process resumes in the state in which it was interrupted. Because the state is restored exactly, the process cannot detect that it was interrupted.

If no process is available for execution, an *idle process* is selected. This system process consumes unused processor time within the operating system.

Scheduling strategies

To determine which process should receive processing time during a process switch, the scheduler uses a so-called *scheduling strategy .* A scheduling strategy searches the set of processes running in the operating system for...

Machine Translated by Google

Specific criteria determine which process should be executed next. A scheduler typically has several different scheduling strategies available, from which one can be chosen arbitrarily. In this experiment, the strategies *Random, Even, RunToCompletion, RoundRobin ,* and *InactiveAging* are considered as possible strategies. For each strategy, the idle process should only be selected if no other process is waiting to be executed. This is particularly relevant if a process is started after no other process has been active. The strategies used in SPOS are presented below.

should:

The **Even** strategy goes through the selectable processes in ascending order, so that each process is selected equally often and the last selectable process is followed by the first one again.

The Random strategy **randomly** selects one of the selectable processes according to the discrete uniform distribution in constant runtime.

The **RunToCompletion** *strategy* selects the currently running process until it terminates. Then, the next selectable process is chosen, analogous to the Even strategy. In this trial, programs do not yet terminate, but this will be possible in later trials.

In the **Round** Robin strategy, the scheduler manages a time slice for each currently running process. This time slice is initialized with the process's priority and decremented with each subsequent scheduler call. This selects the process a certain number of times, until the time slice reaches zero. At this point, the scheduler selects the next available process, similar to the *Even* strategy.

In this strategy, in addition to priority, each process is **assigned** an age, which is initialized to 0. With each scheduler call, the age of each process increases by its priority. The oldest process is always selected as the active process, and its age is reset. However, the other, inactive processes continue to age, ensuring that even a low-priority process will eventually be selected due to its long wait time. This strategy can be described step by step:

1. The age of all processes is increased by their priority.
2. The oldest process is selected and its age is reset to 0.
   If multiple processes with the same age exist, the highest-priority one is selected. If multiple processes with the same age and priority exist, the process with the lowest process ID is selected.

The *InactiveAging* strategy is illustrated in Table 2.1 using the example of three processes with priorities 2, 5, and 17. The first three columns show the age of processes 1-3, with their priority also indicated in the table header. The last column always shows the process selected by the strategy. In the first row...

Machine Translated by Google

1 [= 2]	2 [= 5]	3 [= 17]	Selected process
0	0	0	-
2	5	17 ÿ 0	3
4	10	17 ÿ 0	3
6	15	17 ÿ 0	3
8	20 ÿ 0	17	2
10	5	34 ÿ 0	3
12	10	17 ÿ 0	3
14	15	17 ÿ 0	3
16	20 ÿ 0	17	2
18	5	34 ÿ 0	3
20 ÿ 0	10	17	1
2	15	34 ÿ 0	3
4	20 ÿ 0	17	2
6	5	34 ÿ 0	3
8	10	17 ÿ 0	3
10	15	17 ÿ 0	3
,	,	,	,

Table 2.1: Illustration of the InactiveAging scheduling strategy

The state is visible immediately after initialization, even before the first scheduler call. The following lines contain the age values at the time the next process is selected. An expression of the form 17 ÿ 0 means that the corresponding process has been selected and its age is therefore reset to 0.

Learning success questions

- What distinguishes a process from a program?
- Assume that Table 2.1 contains a fourth process with priority 85. How often
  Would this be executed in a maximum number of consecutive cycles?
- What is the idling process needed for?

Machine Translated by Google

Process flow without critical areas. Process flow with critical areas.

Process 1	Process 2
lcd_writeChar('H');	
lcd_writeChar('a');	
	lcd_writeChar('W');
	lcd_writeChar('e');
lcd_writeChar('l');	
lcd_writeChar('l');	
	lcd_writeChar('l');
	lcd_writeChar('t');
lcd_writeChar('o');	

Edition: HaWelllto

Process 1	Process 2
os_enterCriticalSection();	
lcd_writeChar('H');	
lcd_writeChar('a');	
lcd_writeChar('l');	
lcd_writeChar('l');	
lcd_writeChar('o');	
os_leaveCriticalSection();	
	os_enterCriticalSection();
	lcd_writeChar('W');
	lcd_writeChar('e');
	lcd_writeChar('l');
	lcd_writeChar('t');
	os_leaveCriticalSection();

Edition: HelloWorld

Figure 2.1: Critical areas for avoiding process interlinking

2.1.3 Critical areas

Operating systems possess many different resources. These can include, for example, processing time, data structures, interfaces, or devices such as printers or monitors. Processes share access to these resources. There are operations in which a process must use a resource exclusively for a certain period of time. This means that no other process may access it during that time.
Such use, or rather the duration of this use, is considered a *critical area.*
These are designated as critical areas. Typically, they are used if the simultaneous use of a resource by multiple processes would result in an inconsistent state of the system.

Figure 2.1 shows an example of an inconsistent state that arises from two processes accessing the LCD simultaneously. In a process flow without critical sections, both processes want to use the LCD to display text.
For simplicity, it is assumed that the scheduler always switches the active process after outputting two letters. The resulting output is inconsistent in that it causes an unexpected and unintended *interlocking.*
Both strings are processed. In the process flow with critical areas, before the text is output, both processes enter a critical area that locks the LCD resource for all other processes and only releases it again when the respective critical area has been exited.

2.1.4 Stack

A stack (or stack storage) stores data according to the LIFO (Last-In-First-Out) principle. This means that the byte that was stored most recently is read first. Operations on the stack are typically performed using...

Machine Translated by Google

The stack operates using *push* (placing a new byte on top of the stack) and *pop* (reading and removing the topmost byte from the stack). Access to a stack is not explicitly via *push* and *pop,* but rather via a *stack pointer,* which points to the first free memory location and is incremented or decremented accordingly. When a new byte is placed on the stack, it is stored in the memory location pointed to by the stack pointer. The stack pointer is then moved to the next smallest (free) memory location. Figure 2.2 shows an example of a stack with a maximum capacity of eight bytes and a current fill level of four bytes.

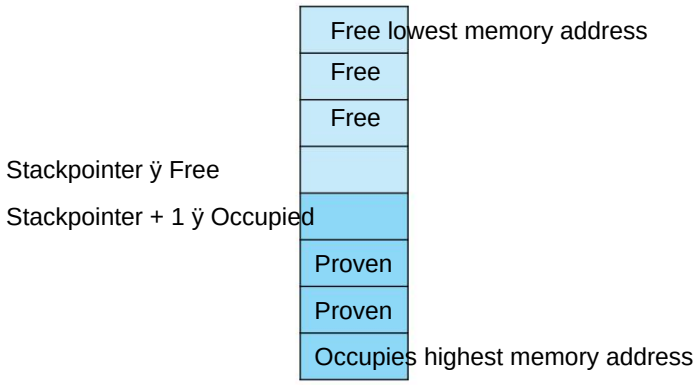


Figure 2.2: Example of a stack

Process stacks

In an operating system, stacks in the form of *process stacks* are used to store local variables and return addresses. The operating system manages one of these stacks for each process, thus enabling it to have an independent program flow.

The processor register PC *(Program Counter)* is a pointer to the next instruction to be executed by the processor. When switching to a subfunction, the current value of the PC register is placed on the stack of the running process and thus temporarily stored, allowing the process to resume from the same point when exiting the subfunction. The address of the first instruction of the called method can then be written to the PC register to execute it. During the execution of this subfunction, any local variables are stored on the stack and removed towards the end of the function. The compiler generates these instructions by analyzing when local variables were last used, usually directly in the machine code. After the last instruction of the subfunction, all local variables are removed, and the topmost entry on the process stack is the return address of the process, which was placed on the stack during the jump. After each return or end of a void function, the compiler places instructions that remove the return address from the stack and load it back into the PC register.

This enables nested function calls: The stack *grows* with each iteration.

Machine Translated by Google

jump into functions, and *breaks* down again during the unwinding of the nesting, the jumps back .

Stack Overflows

It's possible for a process's stack to grow beyond its intended size due to a large number of nested subfunctions or the definition of many local variables. The values placed on the process stack in this way may overwrite another process stack or data area. This is called a *stack overflow* and belongs to the group of *stack inconsistencies.*

Learning success questions

- How is data stored on a stack?
- What does LIFO mean?
- Why is it necessary to provide each process with its own process stack?
  instead of using one stack for all processes?
- What (mal)behavior of the program is possible when a stack overflow occurs?
  is kicking?

Heuristics for detecting stack inconsistencies

The consistency of a process stack can be determined using a *checksum :* If a process causes a stack overflow and overwrites data on the process stack of another process, the checksum of the data in the memory area of the overwritten process stack is highly likely to change.

To ensure the consistency of a process stack, its checksum can be generated when the associated process is interrupted by the scheduler.
Before the scheduler allocates processing time to this process again, the checksum for its process stack is recalculated. If the checksum is identical to the value calculated during the last interruption, the process stack has most likely not been overwritten by another process. If the checksum does not match the previously calculated value, the data on the process stack has been overwritten by another process in the meantime. The process stack is therefore inconsistent.

Machine Translated by Google

Learning success questions

- When does a stack overflow occur?
- What does *inconsistent* mean in the context of process stacks?
- How can a stack overflow be detected?
- Can detected stack inconsistencies be resolved?

2.1.5 Memory allocation

The microcontroller's SRAM memory is divided into several sections, as shown in Figure 2.3. Global variables, a heap, and the system and process stacks are stored in this memory. Half of the SRAM is allocated to the stacks, while the other half is divided between the global variables and the heap. The heap will be discussed in Experiment 3, while the following describes the precise allocation of memory space for the stacks: The SRAM stack area is divided into the area for the system stacks (the stack containing the main function and the scheduler) and the area for the process stacks. The area for the process stacks is divided equally into as many individual process stacks as the maximum number of concurrently running processes allows. The starting addresses of the remaining stacks can then be calculated from the size of the individual memory areas and the starting address of the first stack.

! Danger

The microcontroller's SRAM is addressed starting at 0x0100 . It shares the address space with the processor and I/O registers.

2.1.6 Storing Strings

Constant strings consume a lot of memory. Since the microcontroller's SRAM is already largely reserved for the heap and process stacks, it is not practical to store many additional constant strings there.
It is possible to store this data in the microcontroller's significantly larger flash memory by adding the compiler attribute PROGMEM to the data type when declaring a variable . To store strings in flash memory, a `#define` can be used, which is available in the `avr/pgmspace.h` library . The definition `PSTR(str)` designates the specified string `str` for the compiler to store in flash memory. Such a string can be displayed on the LCD using the method ` void lcd_writeProgString(const char *)` . Its use is demonstrated in Listing 2.1.

Machine Translated by Google

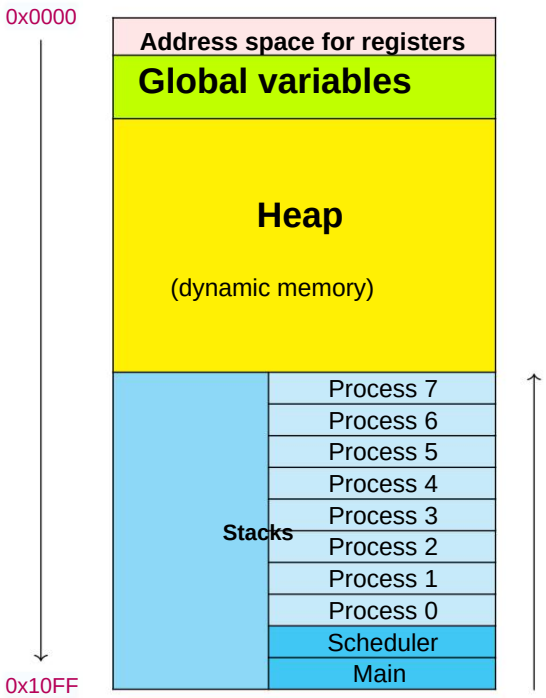


Figure 2.3: Memory allocation

```
1  void example() {
2    // String stored in SRAM
3    lcd_writeString("Hello World");
4
5    // String stored in Flash
6    lcd_writeProgString(PSTR("Hello World")); }
7
```

Listing 2.1: Using PSTR(str)

Machine Translated by Google

2.2 Homework

Instructions for homework

Complete all the tasks presented here and submit your solutions by the deadline in the Moodle learning platform . Your submission should include:

- All .c/.h files
- All project files: .atsln, .xml and .cproj
- Makefile

No test tasks, PDFs, or Doxygen files should be submitted.

! Danger

All functionalities required in the experiment must be implemented!

! Danger

Your code must compile without errors or warnings using test tasks and enabled compiler optimizations. How to enable these optimizations is described in section *6.3.2.*

This information can be found in the *accompanying document* .

To check for warnings, use the "Rebuild Solution" command in the "Build" dialog.
Menu of Microchip Studio 7. The other buttons displayed in the graphical interface only perform incremental compilation, meaning only changed files are recompiled. Warnings and error messages in unchanged files are not displayed.

2.2.1 System Programming Operating System – SPOS

The core tasks of a multitasking operating system include the management of programs and processes, process scheduling, and the management of available memory and other resources.

Machine Translated by Google

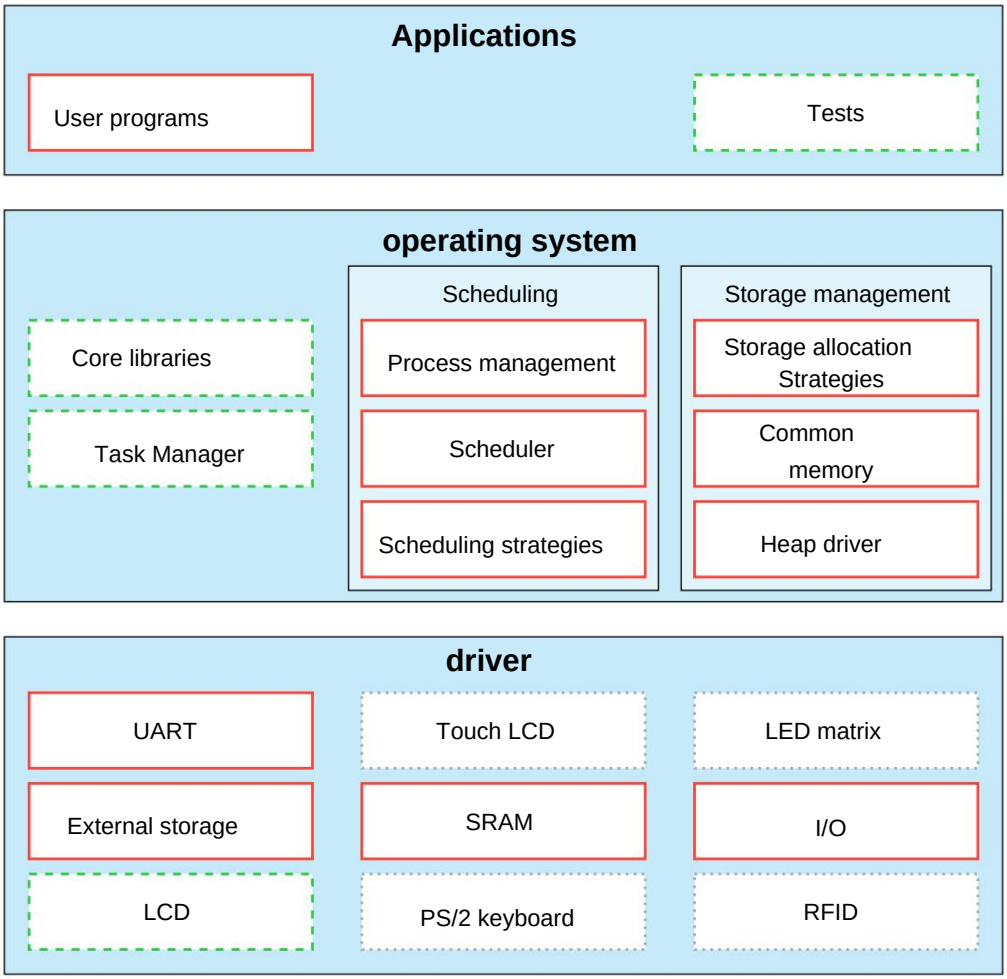


Figure 2.4: Layered model of the SPOS operating system

This and subsequent experiments deal with the implementation of the SPOS operating system for the evaluation board known from Experiment 1. Figure 2.4 visualizes the architecture of the operating system. The core of SPOS comprises scheduling and memory management, which are based on the core libraries. Furthermore, SPOS is extended by drivers that, for example, enable the control of an LCD.
Applications running within SPOS can utilize functions provided by the system.

Components marked with green dashed lines in the diagram are predefined. Components with a red border will be implemented during the course of the practical exercise. One of the drivers marked with gray dots will be part of the final experiment.

Machine Translated by Google

The code framework

To reduce the implementation effort, a code framework is available for this experiment, in which some basic functions are already implemented and which contains approaches for the components presented in this experiment.

Notice

Familiarize yourself with the code framework before working on the homework.
Use the included functions and #defines from the files defines.h, atmega644constants.h and util.h whenever possible to avoid potential errors.

! Danger

Your implementation of this and all subsequent experiments builds upon one another. Therefore, it is important to work carefully and adhere precisely to the specifications in the experiment description.

2.2.2 Error handling

Create a function that conveniently outputs critical errors that may occur during the execution of SPOS.

To implement error output, the function `void os_errorPStr(const char *)` must be implemented in the file `os_core.c` . It should display the passed error message on the LCD and temporarily halt the operating system by globally disabling interrupts. This is achieved by disabling the 7th bit (MSB) in the status register (SREG) .

Before displaying the error, the LCD should be cleared using the `void lcd_clear(void)` method so that the output is not affected by existing text. The error itself should be displayed on the LCD using `void lcd_writeErrorProgString(const char *)` . The user should be able to acknowledge the error message by pressing Enter + ESC , and the operating system should resume once all keys are released. To query the button status, you can use the `uint8_t os_getInput(void)` method , which you implemented in Experiment 1.

Before exiting os_errorPStr , only the 7th bit of the SREG should be reset to the same state as before entering the method. No further action is required regarding the error.

It is entirely possible that SPOS will no longer behave correctly after a manual error acknowledgement. However, in most cases, undefined behavior should be avoided (where possible), as certain test tasks intentionally provoke errors to test the correct behavior of different implementations.

Machine Translated by Google

In some parts of your implementation, it makes sense to catch certain errors and display them to the user. Generally, you should not catch errors that can be handled. For information on displaying error messages, also refer to section 2.1.6. For this purpose, use the macro ` os_error` , which is already defined in the code framework, as shown in Listing 2.2, and not `lcd_writeErrorProgString`.

```
1 os_error("Error!"); // os_errorPStr(PSTR("Error!"));
```

Listing 2.2: Using os_error

2.2.3 Scheduler in SPOS

For the periodic execution of the scheduler described in section 2.1.2, a so-called *timer interrupt* is used, which is triggered regularly and calls an *Interrupt Service Routine* (ISR). This ISR is designed to interrupt the execution of a process, save its state, select a new process, and resume its execution. The state of a process includes the following data:

- Program: Pointer to the program that is instantiated by the process
- Process state: The readiness status of the process (e.g., Ready, Running, …)
- Process priority: The priority of the process relevant to the scheduler.
- Stack pointer: Pointer to the next free memory cell of the process stack process
- Program counter: The address of the currently executed instruction
- Runtime context: The current contents of the processor registers and the status register SREG

The process data— *program, process state ,* and *process priority*— are managed in a *process array .* Additionally, the stack pointer for a waiting process is stored there . The *program counter* and *runtime context* are also stored there.
If a process is waiting to execute, we place the request for the request on its process stack. The functionality of a stack is described in section 2.1.4.
If the process is currently running, the current program counter is located in the PC register, and the runtime context is in the other processor registers designated for this purpose.
Furthermore, the scheduler should include an implementation of critical areas, as described in section 2.1.3. To ensure the consistency of the individual process stacks, SPOS should provide a heuristic for detecting stack inconsistencies.

Machine Translated by Google

2.2.4 Management of programs and processes

Implement the functions, data types, and variables described below for managing programs and processes. Reading the foundational chapter 2.1.1 is required to complete this exercise.

Programs

Programs in SPOS are defined as parameterless functions without a return value. To do this, define the type name `typedef void Program(void)` in `os_process.h` .

Processes

Processes can be understood as instances of programs.

1. In the file os_process.h, implement the structure `struct Process`, which, as indicated in section 2.2.3, combines the process data: *program, process state, process priority ,* and *stack pointer .* Use the `ProcessState` enum as the data type for the process state and the ` sp` union for the stack pointer . The stack pointer value should be retrievable as a `uint16_t` , but direct access to the referenced memory should also be possible. A union allows this without the need for regular value conversion. Further information on the use of ` union` and its distinction from `struct` can be found in the accompanying document or in Experiment 0.

   The data type for the program ` program` is a pointer to the type name `Program` that you defined . For the process priority ` priority` , use the type name ` Priority` for the type ` uint8_t`. The structure has already been named ` Process` using `typedef` to make it more readable and convenient to use later.

2. To allow the scheduler access to all processes, the process array ` os_processes` must be created globally in the file `os_scheduler.c` . This array should contain exactly MAX_NUMBER_OF_PROCESSES entries of type Process . The entries in this array are called *process slots .* The indices of the processes in the `os_processes` array correspond to the process IDs (PIDs) of the processes.

2.2.5 From Program to Process

To start a program, a new process is created as an instance of that program, and the relevant process information is stored. This is done using the function `ProcessID os_exec(Program *, Priority)` , which is defined in the code framework and needs to be extended. The function's task is to store the necessary information about the newly created process in the first free slot of the `os_processes` array and to prepare the required stack memory for the process. To do this, `os_exec` first performs the following steps:

1. Finding a free slot in the process array: The first free slot in os_processes should be used for the newly created process. If all slots are occupied, os_exec should ...

Machine Translated by Google

Return the value INVALID_PROCESS . Whether a slot is already occupied can be determined from the process state.

2. Checking the Program Pointer: A possible error scenario would be, for example, passing a NULL pointer. NULL is already defined in defines.h and can be used directly. If the value NULL is returned, the function execution should be aborted with INVALID_PROCESS as the return value.

3. Saving the Program, Process State, and Process Priority. The process state indicates the current status of a process. When a new process is created, it should initially receive the state OS_PS_READY . The program must also be saved, just like the process state, so that it is possible to associate the running process with the program it is executing. Additionally, the process priority must be saved to enable the application of prioritizing scheduling strategies.

4. Preparing the Process Stack As explained in the fundamentals section on *process stacks* , return addresses reside on the stack. Starting a process is treated like returning from a subfunction. A 16-bit program address is loaded from the stack into the PC register. To start a process, the corresponding initial data must therefore be pre-stored on its process stack. This data includes the initial return address and, since we temporarily store the runtime context on the stack for interrupted processes, the initial runtime context.

To write to the stack, use the union StackPointer and the #define PROCESS_STACK_BOTTOM(PID) from the defines.h file to specify the first (highest)
The storage location of the process stack can be found via the PID.
The program pointer is used as the program counter to start a process. The low byte of the 16-bit address must be written to the stack first. The high byte is then written at the next lower byte position. The runtime context consists of the 32 processor registers and the status register SREG, all of which must be initialized with byte 0. Therefore, 33 zero bytes are written to the process stack after the function pointer. Finally, the stack pointer of the new process must be set to the first available memory location on the process stack.

Once all these steps have been completed, the new process is available to the scheduler. Figure 2.5 shows an example of the process stack after executing os_exec , where process slot 1 is the first free slot in the process array os_processes.
was found. Note that os_exec should not display any error messages.

Review of your implementation

You can use the Unittest os_exec test task to verify your implementation of the os_exec function . Before testing, read section 2.3.1 on test tasks.

Machine Translated by Google

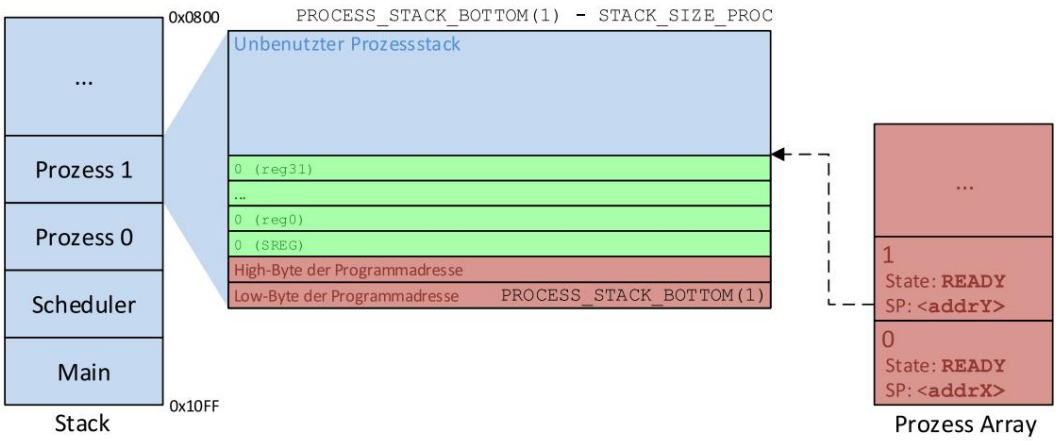


Figure 2.5: Process stack 1 after execution of os_exec

2.2.6 Implementation of the Scheduler

This section describes the implementation of the scheduler ISR, which is called regularly by a timer pre-configured in the code framework to give the user the impression that processes are running concurrently. This ISR is designed to pause the execution of a process, save its state, select a new process, and resume its execution.

Idle process

In the file os_scheduler.c, extend the program void idle(void) so that it continuously outputs dots '.' to the LCD in an infinite loop and waits for the number of milliseconds specified by the define DEFAULT_OUTPUT_DELAY . You can use the delayMs(int ms) function to implement the appropriate delay.
Do not use lcd_clear() between the dots.

Scheduler initialization

The macro REGISTER_AUTOSTART(program) from os_process.h inserts programs into the linked list autostart_head , which is used to automatically start the contained programs at system startup. The function os_initScheduler, present in the code framework , is intended to initialize the process array at operating system startup so that a process is instantiated for each program listed in autostart_head .

Prepare the `os_processes` array by setting the `state` field of all entries to `OS_PS_UNUSED` . Ensure that the idle program ` idle` (see section 2.2.6 *Idle Process)* is not included in the `autostart_head` list.
is located, but should be explicitly started as a process with PID 0. Then iterate through the linked list autostart_head, which is populated with the following before SPOS is executed:

Machine Translated by Google

The necessary program pointers have been initialized. Programs in the list should be started using the `os_exec` function and the `DEFAULT_PRIORITY` priority. The end of the list or an empty list has the value NULL.

Review of your implementation

You can use the Unittest os_initScheduler test task to verify your implementation of the os_initScheduler function . Before testing, read section 2.3.1 on test tasks.

Notes on debugging

An incorrectly initialized or restored stack will most likely cause the microcontroller to crash ("software reset"). For example, an incorrect return address (e.g., NULL) can lead to a jump to a location in program memory containing inappropriate code or no code at all. The processor then enters a state where program execution is no longer possible, and the microcontroller restarts.



Figure 2.6: LCD output during a software reset

The `os_init` function, defined in the code framework , displays the reason for the restart (see, for example, Figure 2.6). The message must then be acknowledged. Other possible reasons for a reset include:

JT - JTag Reset The JTag triggered the restart, for example by flashing new code.

EXT - External Reset The restart was initiated by a logical 0 at the reset pin, for example by pressing the reset button on the evaluation board.

POW - Power Reset, BO - Brownout Detection Reset. A faulty power supply to the microcontroller is the reason for the restart.

Saving the current process

The currently running process must be queried at any time by calling the function `ProcessID os_getCurrentProc(void)` . To achieve this, create a global variable named `currentProc` at a suitable location and implement the function `os_getCurrentProc` with an appropriate return value.

Machine Translated by Google

Implementation of the Scheduler ISR

The scheduler ISR TIMER2_COMPA_vect, present in the code framework , is automatically called by a timer at intervals of approximately 3 ms. Extend this ISR to enable switching between different processes.

When a process is interrupted, its runtime context must be saved on the stack. Similarly, the current stack pointer must be saved by the scheduler so that the process can resume from the same point later. Note that the ISR is declared as " naked ." This means that the compiler should not save any process registers or allocate any memory for local variables when calling the function. Therefore, the use of many local variables within the ISR itself should be avoided, as this can lead to undefined behavior.

Notice

Do not use local variables in the ISR and use #define BOTTOM_OF_ISR_STACK to set the stack pointer to the scheduler stack .

Switching between process stacks: The current stack pointer used by the processor is stored in register SP1 . Register SP contains the address of the first free memory location of the currently used process stack. To switch between two process stacks, the contents of register SP are first stored as the stack pointer of the current process. The union StackPointer, defined in the code framework, already exists for this purpose. Register SP is then set to the stack pointer of the next process. Subsequent stack operations then refer to the process stack of the next process.

Scheduler process and process change: The scheduler process and the associated process change comprise the following steps:

1. Storing the program counter as the return address for later resuming the current process on its process stack. This is done implicitly when jumping to the ISR.

2. Saving the runtime context of the current process to its process stack.
   This is done using the assembler macro saveContext().

3. Saving the stack pointer for the process stack of the current process.

4. Setting the SP register to the scheduler stack.

5. Setting the process state of the current process to OS_PS_READY.

---

1. The SP register, like the DDRA register, can be accessed directly from the program code. If Microchip Studio 7 displays errors here with red underlined code, you can ignore this; compilation should still work.

Machine Translated by Google

6. Selection of the next process to be continued by calling up the currently active process.
   applied a scheduling strategy.
7. Setting the process state of the process to be continued to OS_PS_RUNNING.
8. Restore the stack pointer for the process stack of the process to be continued
   process.
9. Restore the runtime context of the process to be continued by the
   Assembly macro restoreContext().
10. Automatic return to the original text using the assembler macro restoreContext().

Figure 2.7 illustrates the state of SPOS before the process switch, with the stack on the right, the microcontroller registers at the top, and the process management data structures on the left. Figures 2.8 and 2.9 illustrate steps 1 to 3 and 8 to 10 of the scheduler, respectively, using an example process switch from process 1 to process 2. The position of the stack pointer is shown by a dashed line, and value assignments by a solid line. Note that after each push or pop stack operation , the stack pointer is decremented or incremented, and Figure 2.8 shows the stack pointer after step 2, while Figure 2.9 shows it before step 9. The supplementary document " *Scheduler Flow,* " which you can find in the Moodle learning space, provides further information.

Finding represents the scheduler process step by step, with the position of the stack pointer updated after each step.

Once these operations have been completed, the resumed process has access to the runtime context it had before the scheduler interruption. Figure 2.10 illustrates this using the example above. Execution resumes from where it was previously interrupted by the scheduler because the registers and program counter previously placed on the stack have been restored. The context of the ISR (and thus the value of the stack pointer at the end of the ISR) does not need to be stored, since an ISR does not expect to find data on the stack. It is therefore perfectly correct to reset the stack pointer to the beginning of the scheduler stack with each invocation of the ISR.

Starting the scheduler

Implement the function `void os_startScheduler(void)`. The steps to be performed are:

1. Setting the variable for the current process to 0 (idle process).
2. Change the idle process state to OS_PS_RUNNING .
3. Setting the stack pointer to the process stack of the idle process.
4. Jump to the idle process using restoreContext().

Machine Translated by Google

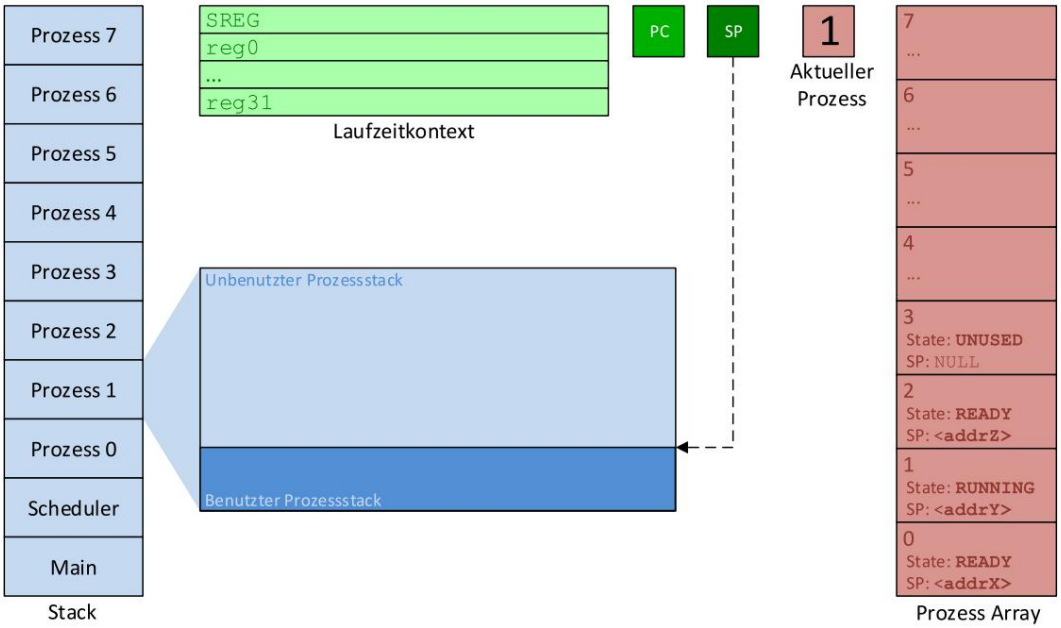


Figure 2.7: State before process change from process 1 to 2

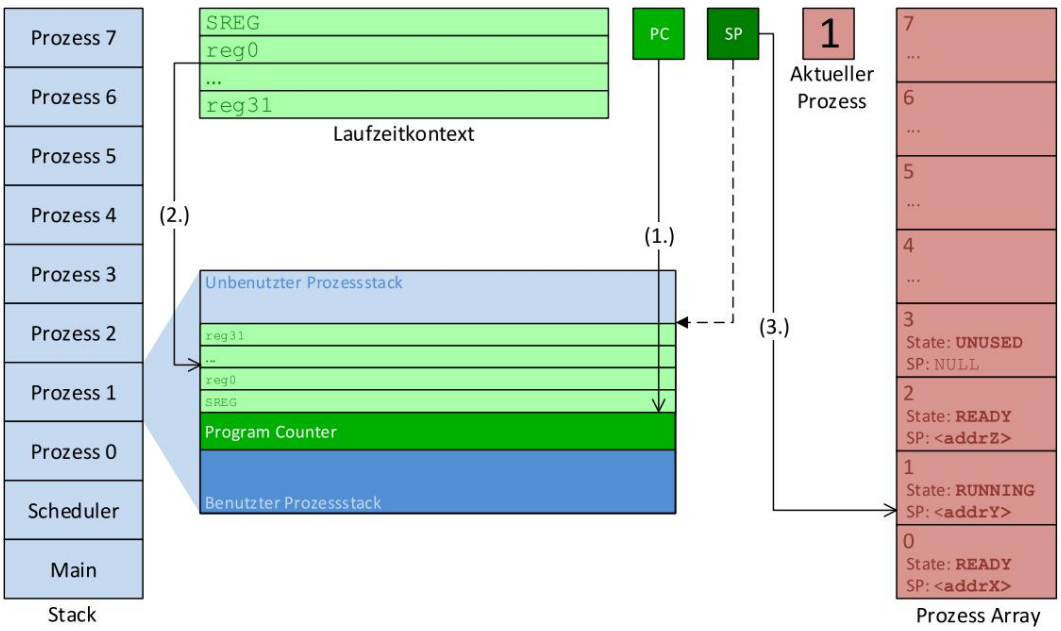


Figure 2.8: Steps 1-3 of the process change from process 1 to 2

Machine Translated by Google

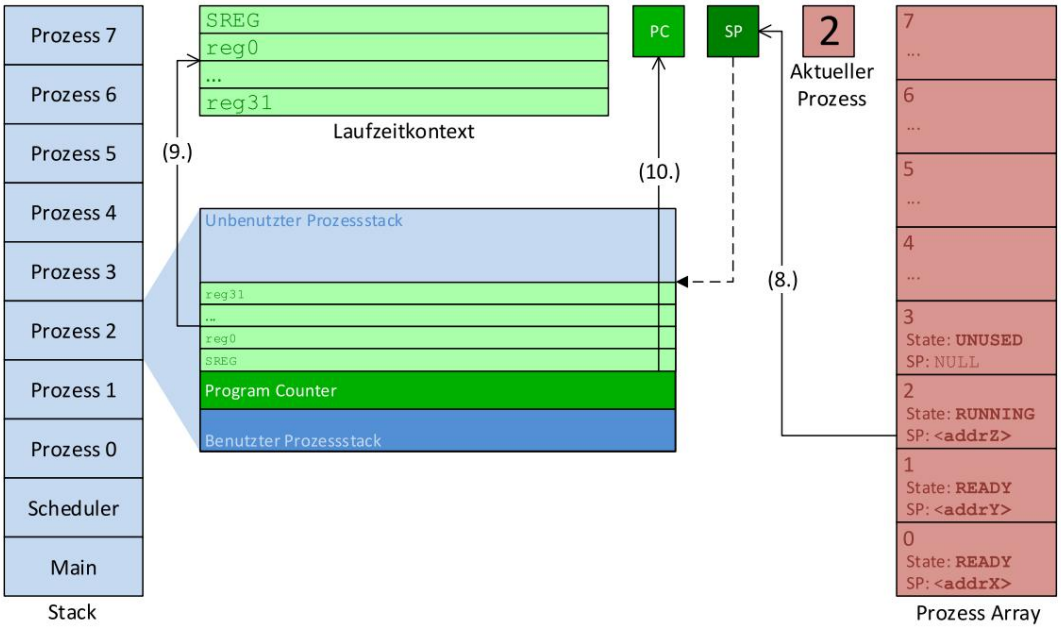


Figure 2.9: Steps 8-10 of the process change from process 1 to 2

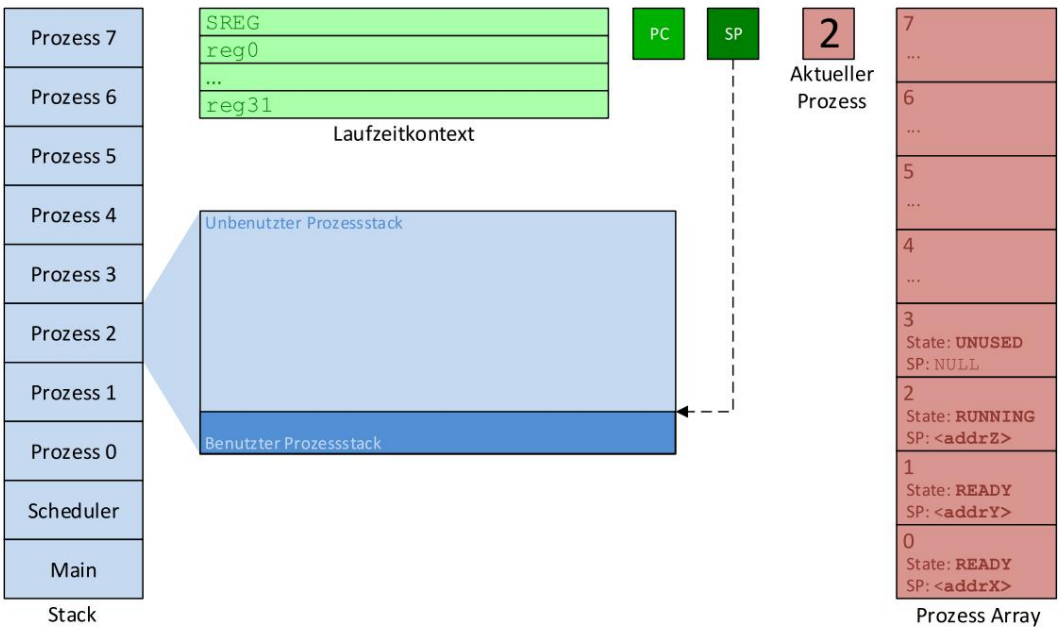


Figure 2.10: State after process change from process 1 to 2

Machine Translated by Google

Scheduling strategies

To allow selection between multiple scheduling strategies, a global variable should be created to store the currently used scheduling strategy. Read and write operations should be performed using the functions `SchedulingStrategy os_getSchedulingStrategy(void)` and `void os_setSchedulingStrategy(SchedulingStrategy)` . These functions are already present in the code framework and need to be added. Use the `SchedulingStrategy` enum type to store the current scheduling strategy .

The possible strategies considered in this experiment are the *Random* and *Even* strategies. Considered. The implementation of the strategies *RunToCompletion, RoundRobin,* and *InactiveAging* is an optional in-person task for this experiment. All scheduling strategies should select only processes that are in the OS_PS_READY state.
The idle process should only be selected in all strategies if no other process is available.

Implement the scheduling strategies *Random* and *Even.*

Use the rand() function to generate a random number between 0 and 32767.
Care should be taken to ensure that the `rand()` function is called only once and that the selection of processes is evenly distributed. Therefore, proceed in such a way that your implementation, if the ` rand()` function returns 0,
Returns the process with the smallest PID of all processes in the OS_PS_READY state. If the return value is 1, the process with the second smallest PID is returned, and so on. If the return value is higher than the number of processes in the OS_PS_READY state, the process starts again from the beginning. The modulo operator % can be used to achieve this behavior .

2.2.7 Implementation of critical areas

Section 2.1.3 introduced the concept of critical sections. To manage critical sections in SPOS, the functions `void os_enterCriticalSection(void)` and `void os_leaveCriticalSection(void)` should be implemented in `os_scheduler.c` . These functions should enter and exit a critical section by switching the scheduler off and on again, respectively. These functions must be *atomic* and therefore must not be interrupted by interrupts.

To disable the scheduler in the os_enterCriticalSection function, the OCIE2A bit in the TIMSK2 register must be set to 0. Clearing this bit prevents an interrupt for TIMER2_COMPA_vect, and therefore no scheduler call.

Since further critical sections can be called within subfunctions, it must be possible to nest critical sections within each other. To do this, create the global variable `criticalSectionCount` to store the nesting depth. The process when entering `os_enterCriticalSection` should follow these steps.
orient:

Machine Translated by Google

1. Storing the Global Interrupt Enable Bit (GIEB) from SREG2 in a local Variable.
2. Disabling the Global Interrupt Enable bit to enable atomic behavior of the To ensure functionality.
3. Increment the nesting depth of the critical area.
4. Deactivate the scheduler.
5. Restore the (previously saved) state of the Global Interrupt Enable Bit in SREG.

The function `os_leaveCriticalSection` should be implemented analogously to `os_enterCriticalSection` , with the difference that the scheduler is only reactivated if, after decrementing the nesting depth, it reaches the value 0. Thus, the scheduler is only reactivated after all nested critical sections have been exited. Implement error checks at appropriate points within this function. For example, it is an error if `os_leaveCriticalSection` is called more often than `os_enterCriticalSection`. In the event of an error in `os_enterCriticalSection` or `os_leaveCriticalSection` , `criticalSectionCount` should not be modified.

Integration of critical sections

After implementing the functionality described above, extend your implemented os_exec function with a critical section that protects the existing implementation from interruption by the scheduler.

Review of your implementation

At this point, the basic components of the operating system are functional. You can use the Error, Critical, Multiple , and Resume test tasks to check your work so far. Explanations of the individual test tasks can be found in the test task description available in the Moodle learning space .

2.2.8 Detection of Stack Inconsistencies

Extend the previously created scheduler ISR TIMER2_COMPA_vect to include the ability to detect stack inconsistencies. To do this, complete the function StackChecksum os_getStackChecksum(void) in os_scheduler.c.

A heuristic for detecting process stack inconsistencies should be implemented. To calculate the checksum, all bytes of the process stack should be XORed bitwise. This check function has the advantage of requiring little computational effort, and therefore calls to this function are relatively short.

---

2. Further information on the SREG register can be found in the *ATmega644 datasheet.* in Chapter 5.4.

Machine Translated by Google

The checks can be processed in a timely manner. The StackChecksum type is already predefined with a suitable value range. It is sufficient to include in the check the data that lies between the beginning of the process stack and the stack pointer of the corresponding process. Data that lies beyond this stack pointer is not used by the process and should not be considered.

Integration into the operating system

Extend the `struct Process` structure with the attribute `checksum` of data type `StackChecksum`, which stores a checksum of the process stack. When starting processes using the `os_exec` function, ensure that this checksum is correctly initialized after the process stack has been prepared. Now extend the scheduler so that, after setting the stack pointer to the scheduler stack, it determines and stores the checksum of the interrupted process's stack. A process selected in the scheduler should only resume if the checksum has not changed since the interruption. If it has changed, simply output an error message using `os_error` and then allow the scheduler to continue running normally.

Review of your implementation

You can use the Stack Consistency test task to verify your detection of stackin consistency.

2.2.9 Opening the Task Manager

A task manager for SPOS has already been provided to you in the files os_taskman.c and os_user_privileges.c/.h . The task manager should be invoked from within the scheduler when Enter + ESC are pressed simultaneously. The task manager offers various ways to interact with the operating system. For example, it is possible to start additional programs or change the scheduling strategy. Add the task manager call to your scheduler implementation.

Implementation guidelines

To ensure that the Task Manager is always launched from a defined state and not arbitrarily, it is advisable to perform the key press check after step 4 and before step 5 of the scheduler sequence described on page 20. Note that after pressing the keys, you must wait until all keys have been released. Otherwise, the starting Task Manager would immediately interpret the still-pressed keys as input, which should be avoided.

Machine Translated by Google

Notice

The task manager contains functions that are not yet available in this experiment. It can be adapted to the specific experiment using the `#define EXPERIMENT` directive in the `defines.h` file.

Learning success questions

- What is the difference between atomic and critical areas?
- How are critical or atomic areas created in the code?
- How do you create a new program in SPOS?

Machine Translated by Google

2.2.10 Summary

The following overview lists some functions, types, and tasks relevant to this experiment. It can be used as a checklist. This overview is not guaranteed to be complete.

- [ ] os_core.c/.h
  - [ ] void os_errorPStr(char const *)
- [ ] os_input.c/.h
  - [ ] void os_initInput(void)
  - [ ] uint8_t os_getInput(void)
  - [ ] void os_waitForInput(void)
  - [ ] void os_waitForNoInput(void)
- [ ] os_process.h
  - [ ] struct Process
  - [ ] Process as a type name for struct Process
  - [ ] ProcessID as type name for uint8_t
  - [ ] Program as a type name for void (void)
- [ ] os_scheduler.c/.h
  - [ ] Process os_processes[MAX_NUMBER_OF_PROCESSES]
  - [ ] uint8_t criticalSectionCount
  - [ ] void os_initScheduler(void)
  - [ ] void os_startScheduler(void)
  - [ ] StackChecksum os_getStackChecksum(ProcessID)
  - [ ] ProcessID os_exec(Program *, Priority)
    (incl. critical sections)
  - [ ] ProcessID currentProc

Machine Translated by Google

- [ ] ProcessID os_getCurrentProc(void)
- [ ] ISR(TIMER2_COMPA_vect)
- [ ] void os_enterCriticalSection(void)
- [ ] void os_leaveCriticalSection(void)
- [ ] void os_setSchedulingStrategy(SchedulingStrategy)
- [ ] SchedulingStrategy os_getSchedulingStrategy(void)
- [ ] void idle(void)

- [ ] os_scheduling_strategies.c/.h
  - [ ] ProcessID os_Scheduler_Random(Process const [], ProcessID)
  - [ ] ProcessID os_Scheduler_Even(Process const [], ProcessID)

Machine Translated by Google

2.3 In person

2.3.1 Test tasks

In the Moodle learning space you will find a collection of test tasks that can be used to test the functionality of your operating system at the current stage of development.
The flawless execution of all mandatory test tasks, as described in the test task description also available in the Moodle learning space, will be checked during the first two hours of the in-person session. If problems arise with the test tasks, you have likely not met certain requirements or overlooked specific exceptions. Please supplement or correct your project if problems occur with the test tasks.

! Danger

Even if you are able to successfully execute all the provided test tasks, this does not guarantee that your code is error-free. These test tasks only cover a portion of the possible errors.

2.3.2 Application tasks

During the practical session, you will be given tasks to demonstrate the skills and knowledge you acquired in the experiment. You will have the first two hours of the session to complete these tasks. Since the tasks build upon the content you implemented, it is recommended that you ensure it is functioning correctly (e.g., through test tasks) before beginning.

The tasks are designed to take approximately 30 minutes to complete, assuming a correct implementation beforehand.

! Danger

Further distribution of the tasks, as well as plagiarism or sharing of solutions, is not permitted and will be considered an attempt to cheat, leading to exclusion from the internship.

Machine Translated by Google

2.4 Pin assignment

Port	Pin	Assignment	Port	Pin	Assignment
B	0	-	A	0	LCD Pin 0 (D4)
	1	-		1	LCD Pin 1 (D5)
	2	-		2	LCD Pin 2 (D6)
	3	-		3	LCD Pin 3 (D7)
	4	-		4	LCD Pin 4 (RS)
	5	-		5	LCD Pin 5 (EN)
	6	-		6	LCD Pin 6 (RW)
	7	-		7	-
D	0	-	C	0	Button 1: Enter
	1	-		1	Button 2: Up
	2	-		2	Reserved for JTAG
	3	-		3	Reserved for JTAG
	4	-		4	Reserved for JTAG
	5	-		5	Reserved for JTAG
	6	-		6	Button 3: Down
	7	-		7	Button 4: Escape

Pin assignment 2.1: For attempt 2